Cassandra Cheat Sheet

> Quick reference for exam prep — CQL, architecture, data modeling, operations, and common commands with examples.

Quick overview

- Apache Cassandra: distributed, peer-to-peer NoSQL DB for high availability and linear scalability.
- Key features: tunable consistency, masterless architecture, partitioned data model, wide-column store.

Connect & shell

```
# start CQL shell
cqlsh 127.0.0.1 9042
# set consistency in cqlsh
CONSISTENCY QUORUM;
# enable tracing
TRACING ON;
```

Keyspace (like database)

```
-- create keyspace with replication
CREATE KEYSPACE examks WITH replication = {'class':'SimpleStrategy',
'replication_factor':3};
-- production (multi-datacenter)
CREATE KEYSPACE prodks WITH replication = {'class':'NetworkTopologyStrategy',
'dc1':3, 'dc2':2};
-- use keyspace
USE examks;
```

Table basics (CQL)

```
CREATE TABLE users (
  userid uuid,
  email text,
```

```
  name text,
  created_at timestamp,
  PRIMARY KEY (userid)
);

-- composite primary key (partition key + clustering columns)
CREATE TABLE orders (
  user_id uuid,
  order_id timeuuid,
  product text,
  qty int,
  price decimal,
  PRIMARY KEY ((user_id), order_id)
) WITH CLUSTERING ORDER BY (order_id DESC);
```

Notes: PRIMARY KEY((partition_key), clustering_cols...). Partition key choice determines data distribution.

CRUD (CQL examples)

```
-- insert
essential> INSERT INTO users (userid, email, name, created_at) VALUES (uuid(),
'a@x.com', 'Asha', toTimestamp(now()));

-- update
essential> UPDATE users SET name = 'Asha Rao' WHERE userid = 1234-...;

-- delete
essential> DELETE name FROM users WHERE userid = 1234-...;
DELETE FROM users WHERE userid = 1234-...; -- delete row

-- select
essential> SELECT * FROM users WHERE userid = 1234-...;
-- range query using clustering column
SELECT * FROM orders WHERE user_id = ... AND order_id >
minTimeuuid('2025-01-01');
```

Cassandra queries must include the partition key (unless using ALLOW FILTERING, which is discouraged).

Lightweight transactions (LWT)

```
-- compare-and-set
INSERT INTO users (userid, email) VALUES (uuid(), 'b@x.com') IF NOT EXISTS;
UPDATE users SET email = 'new@x.com' WHERE userid = ... IF email = 'old@x.com';
```

LWTs use Paxos and are slower — use only when strong serializability is required.

Batches & prepared statements

```
-- batch (atomic across partitions only for logged batch)
BEGIN BATCH
  INSERT INTO orders (user_id, order_id, product, qty) VALUES (...);
  INSERT INTO order_by_product (product, order_id, user_id, qty) VALUES (...);
APPLY BATCH;

-- prepared statement (client driver)
PREPARE stmt FROM 'INSERT INTO users (userid, email) VALUES (?, ?)';
EXECUTE stmt USING uuid(), 'x@a.com';
```

Best practice: avoid large batches across many partitions; batches are for atomicity and efficiency on small related updates.

Indexes & Materialized Views

```
-- local secondary index (use sparingly)
CREATE INDEX ON users (email);

-- materialized view (careful: consistency & maintenance cost)
CREATE MATERIALIZED VIEW users_by_email AS
  SELECT * FROM users
  WHERE email IS NOT NULL AND userid IS NOT NULL
  PRIMARY KEY (email, userid);
```

Caveat: secondary indexes and materialized views can cause performance and consistency trade-offs at scale — model data to avoid them when possible.

TTL, writetime, and counters

```
-- TTL: expire a column/row after seconds
INSERT INTO sessions (session_id, user_id) VALUES (...) USING TTL 3600;

-- get write time of a column
SELECT writetime(name) FROM users WHERE userid = ...;

-- counter table
author> CREATE TABLE page_hits (page text PRIMARY KEY, hits counter);
UPDATE page_hits SET hits = hits + 1 WHERE page = 'home';
```

Counters have special semantics and need dedicated tables with only counter columns (and primary key).

Aggregation & limitations

- CQL has limited aggregation (`COUNT`, `MIN`, `MAX`, `SUM`) and they operate per-partition efficiently.
- For global or complex aggregations, use Spark, Spark-Cassandra connector, or an external analytics layer.

Replication & Consistency

- Replication strategies: `SimpleStrategy` (single DC) vs `NetworkTopologyStrategy` (multi-DC).
- Consistency levels (choose per query): `ANY`, `ONE`, `TWO`, `THREE`, `QUORUM`, `LOCAL_QUORUM`, `EACH_QUORUM`, `ALL`, `LOCAL_ONE`.

Read/Write path basics: - Write to commit log and memtable, then hinted handoff if node down, SSTable flush to disk. - Read merges memtable + SSTables; coordinator contacts replicas based on consistency level and reconciles using timestamps.

Example: write with QUORUM and read with QUORUM gives strong consistency (as long as same replica set).

Compaction, Tombstones & Performance

- Compaction strategies: `SizeTieredCompactionStrategy` (STCS), `LeveledCompactionStrategy` (LCS) — good for reads, `TimeWindowCompactionStrategy` (TWCS) — good for time-series.
- Tombstones are markers for deletes; many tombstones cause read amplification and performance issues. Avoid large-range deletes; use TTLs.

- Monitor tombstones with `nodetool cfstats` and tune GCGraceSeconds appropriately when doing repairs.

Operational commands (nodetool & cqlsh)

```
# check node status
nodetool status
# take snapshot
nodetool snapshot -t mysnap keyspace_name
# repair a node
nodetool repair keyspace_name -pr
# compact
nodetool compact keyspace_name table_name
# decommission a node
nodetool decommission
# drain before shutdown
nodetool drain
# view table stats
nodetool cfstats keyspace_name.table_name
```

Backup & restore (quick)

- Snapshot: `nodetool snapshot` creates hardlinks in the data directory; copy SSTables from `data_file_directories/<keyspace>/<table>/snapshots/<snapshot>` to backup storage.
- To bulk load into a cluster: use `sstableloader` to stream SSTables into a live cluster.

Monitoring & tooling

- Metrics: JMX metrics via `nodetool`, Prometheus + Grafana exporters common in production.
- Benchmark: `cassandra-stress` for load testing.
- Logs: system.log for GC and hints; watch for compaction and tombstone warnings.

Data modeling tips (most important for exam)

- Model for queries: design tables per query pattern (query-driven modeling).
- Denormalize and duplicate data — reads are primary and must be fast.
- Choose partition key to keep partitions balanced and not too large (avoid hot partitions).
- Use clustering columns for sorted results within a partition.
- Avoid wide reads that will scan many partitions; avoid ALLOW FILTERING in production.

- Use TimeWindowCompactionStrategy for time-series data.

Examples: common patterns

```
-- time-series events per device (partition by device, cluster by time)
CREATE TABLE device_events (
  device_id uuid,
  event_time timestamp,
  event_type text,
  payload text,
  PRIMARY KEY ((device_id), event_time)
) WITH CLUSTERING ORDER BY (event_time DESC);

-- materialized view to query by email (if needed)
CREATE MATERIALIZED VIEW users_by_email AS
  SELECT * FROM users
  WHERE email IS NOT NULL AND userid IS NOT NULL
  PRIMARY KEY (email, userid);

-- read latest 10 events for a device
SELECT * FROM device_events WHERE device_id = ... LIMIT 10;
```

Troubleshooting quick checklist

- Node down? Check `nodetool status` and logs. Use `nodetool repair` after recoveries.
- High tombstones? Find queries causing range deletes or large partitions, use TTL or rewrite queries.
- Uneven token distribution? Check `nodetool status` and consider rebalancing (bootstrap/decommission) or use vnodes.

Good luck on your exam! If you want this exported as a PDF or shortened to one page, tell me which format and I'll prepare it.